

Аутоаудит баз 1С:Бухгалтерия

Анализ ТЗ, точки роста и архитектура Python-решения

Контекст: Данный документ содержит экспертный анализ предоставленного Технического Задания, предложения по его расширению (до уровня полноценного ИИ-ассистента) и прототип программной логики на языке Python.

1. Анализ ТЗ и точки роста (Что можно добавить)

Текущее ТЗ описывает качественную, но классическую **Rule-based (основанную на жестких правилах)** систему. Она автоматизирует рутину, но пока не является «ИИ-аудитором». Чтобы продукт соответствовал концепции ИИ-помощника, снижающего страховые риски (как обсуждалось в вашей концепции), в ТЗ стоит добавить следующие модули:

1.1. Модуль ML: Поиск аномалий (Anomaly Detection)

Жесткие правила из ТЗ поймут счет 000, но пропустят операцию, которая формально корректна, но статистически подозрительна.

- **Нетипичные суммы:** ИИ должен анализировать историю платежей контрагенту. Если средний чек был 50 000 руб., а новый акт пришел на 5 000 000 руб. — это красный флаг [cite: 1].
- **Резкая смена номенклатуры:** Контрагент всегда оказывал «Услуги по ремонту», а сейчас продал «Строительные материалы».
- **Техническая реализация:** Алгоритмы **Isolation Forest** или **Autoencoders** (Scikit-learn).

1.2. NLP-анализ текстовых полей

Анализ «Назначения платежа» и текстов договоров для выявления налоговых рисков (115-ФЗ, дробление бизнеса).

- Проверка соответствия текста назначения платежа реальному коду ОКВЭД контрагента.
- Поиск триггерных слов (например, неформальные пометки бухгалтера в комментариях).

1.3. Интеллектуальный поиск дублей

Вместо жесткой проверки ИНН, использовать алгоритмы нечеткого поиска (Fuzzy String Matching) для поиска задвоенных контрагентов или номенклатуры (например, «ООО Ромашка» и «Ромашка, ООО»), что часто ведет к развернутому салдо [cite: 1].

2. Архитектура решения на Python

Поскольку продукт должен работать гибко и, возможно, как внешний SaaS-сервис или микросервис для аутсорсеров, писать его прямо на встроенном языке 1С (через внешние обработки) — не всегда лучший выбор для ML-задач. Рекомендуется гибридная архитектура:

- **Слой интеграции (1С OData API):** Python-бэкенд обращается к базе 1С через стандартный протокол OData (REST API), выгружая Оборотно-сальдовые ведомости (ОСВ) и регистры бухгалтерии в формате JSON.
- **Слой данных (Pandas/Numpy):** Данные преобразуются в DataFrame. Pandas позволяет делать проверки миллионов проводок за доли секунды с помощью векторизованных операций.
- **Бизнес-логика:** Применение фильтров согласно ТЗ (Красное сальдо, Счет 000, Незакрытые авансы) [cite: 1].
- **Слой отчетов (Jinja2 / OpenPyXL):** Генерация сводных и детальных отчетов в Excel с подсветкой ячеек красным/желтым/зеленым цветом [cite: 1].

Важно: Для работы с клиентскими базами 1С без выгрузки данных в облако (требование безопасности), Python-скрипт можно упаковать в исполняемый `.exe` файл (через PyInstaller) и запускать локально на сервере клиента.

3. Прототип логики проверок (Python + Pandas)

Ниже представлен базовый каркас класса, который реализует требования из вашего ТЗ для поиска красного сальдо и проверки счета 000 [cite: 1].

```

import pandas as pd

class AutoAuditor1C:
    def __init__(self, balances_df: pd.DataFrame):
        # Ожидается DataFrame с колонками:
        # ['account', 'sub_account', 'debit', 'credit', 'type']
        # type: 'A' (Активный), 'P' (Пассивный), 'AP' (Активно-Пассивный)
        self.balances = balances_df
        self.errors = []

    def check_red_balance(self):
        """ТЗ п. 4.1. Проверка красного сальдо"""
        # Отрицательное сальдо (если в 1С оно хранится с минусом)
        # Или кредитовое сальдо на активном счете
        active_errors = self.balances[
            (self.balances['type'] == 'A') &
            (self.balances['credit'] > 0)
        ]

        # Дебетовое сальдо на пассивном счете
        passive_errors = self.balances[
            (self.balances['type'] == 'P') &
            (self.balances['debit'] > 0)
        ]

        if not active_errors.empty:
            self.errors.append({
                "error_type": "Красное сальдо (Активный счет имеет кредит)",
                "data": active_errors
            })

        if not passive_errors.empty:
            self.errors.append({
                "error_type": "Красное сальдо (Пассивный счет имеет дебет)",
                "data": passive_errors
            })

    def check_account_000(self):
        """ТЗ п. 4.4. Проверка остатков на счете 000"""
        acc_000 = self.balances[
            (self.balances['account'] == '000') &
            ((self.balances['debit'] > 0) | (self.balances['credit'] > 0))
        ]

        if not acc_000.empty:
            self.errors.append({
                "error_type": "Найдено незакрытое сальдо на счете 000",
                "data": acc_000
            })

    def check_unclosed_advances(self, settlements_df: pd.DataFrame):

```

```

"""ТЗ п. 4.5. Незакрытые расчеты с контрагентами"""
# Логика сопоставления отгрузок и оплат...
pass

def run_audit(self):
    self.check_red_balance()
    self.check_account_000()
    return self.generate_report()

def generate_report(self):
    if not self.errors:
        return {"status": "success", "message": "Ошибок не найдено"}

    report = {"status": "warning", "total_errors": len(self.errors),
"details": []}
    for err in self.errors:
        report['details'].append({
            "type": err['error_type'],
            "items": err['data'].to_dict('records')
        })
    return report

```

4. Заключительные рекомендации по UI/UX

Согласно п.7 ТЗ (Пользовательский интерфейс), программа должна быть простой [cite: 1].

Рекомендуем реализовать интерфейс в виде **веб-дашборда (например, на Streamlit или FastAPI + React)**. Бухгалтер просто загружает выгрузку из 1С или нажимает кнопку

«Синхронизировать», а на экране появляются интерактивные карточки с красными флагами. Это гораздо современнее и удобнее, чем стандартный интерфейс внешних отчетов самой 1С.